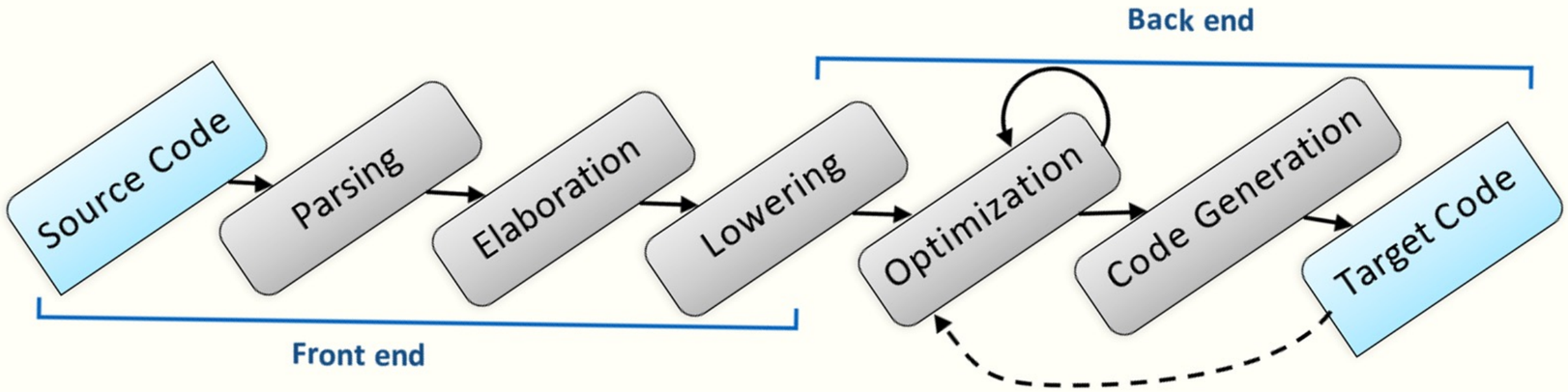
《编译原理》

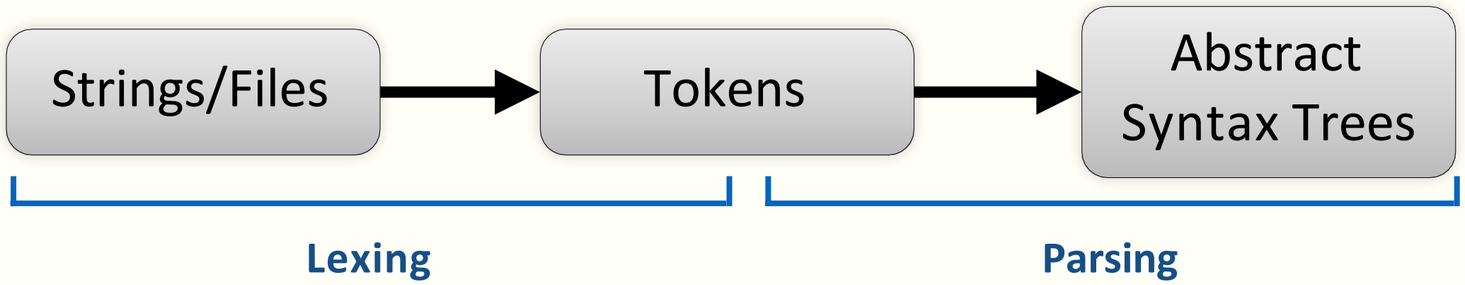
第10讲：生成式前端--从源码到AST

何冬杰
重庆大学

Compiler Architecture

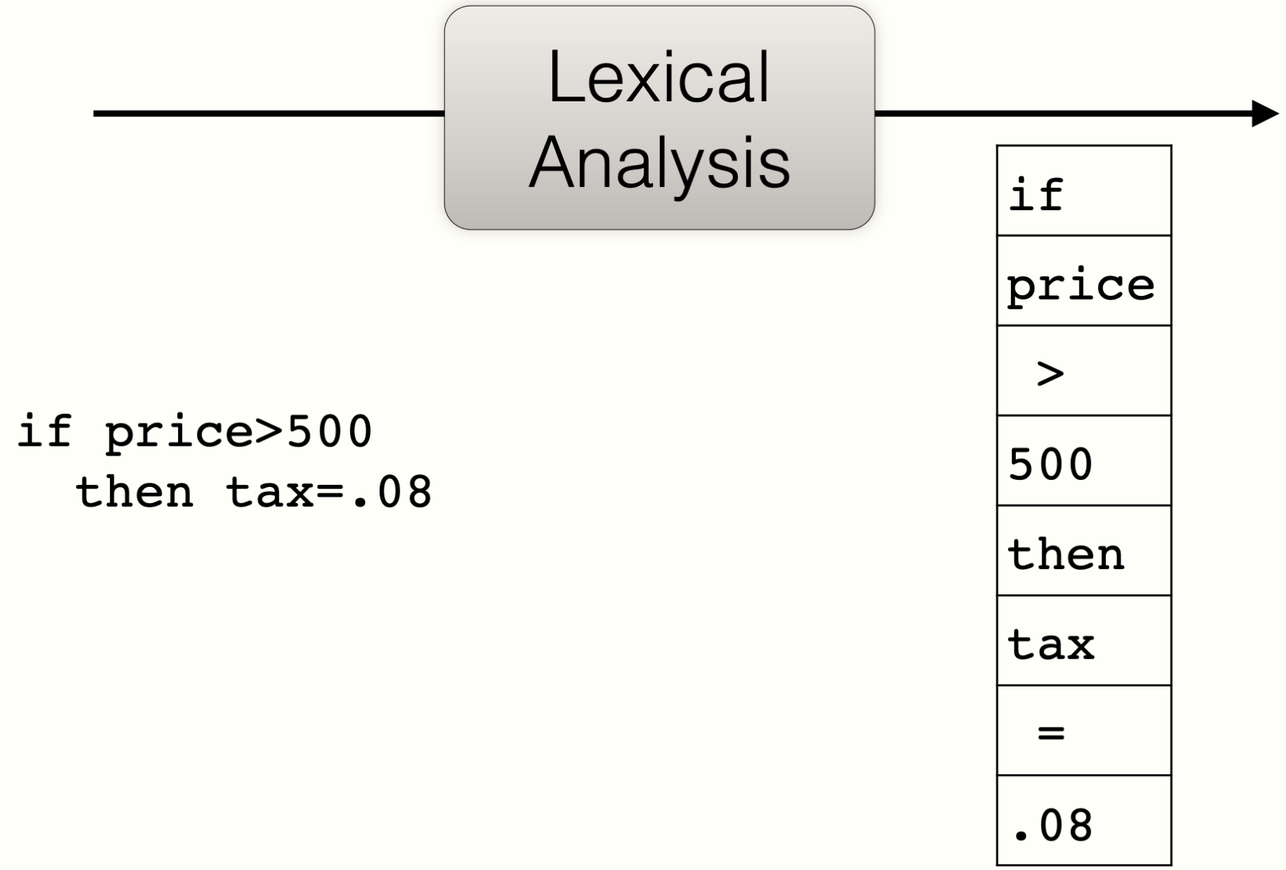


- Front end: Lexing & Parsing
  - From strings to data structures
  - Usually split into two phases:



Lexical Analysis

- breaks input sequence of characters into individual words, aka “tokens”



Lexical Tokens

- A language classifies lexical tokens into token types
  - So, a token type specifies a set of acceptable tokens.
  - Reserved words (E.g., IF, RETURN) are tokens that cannot be used as identifiers

Type	Examples
ID	`foo n14 last`
NUM	`73 0 00 515 082`
REAL	`66.1 .5 10. 1e67`
IF	`if`
COMMA	`,`
NOTEQ	`!=`
LPAREN	`(`

- Example 1: **`if (price>500) then tax=.08`**
  - the lexical analysis returns the sequence of tokens

`IF LPAREN ID(price) GT NUM(500) RPAREN THEN ID(tax) EQ REAL(0.08)`

Lexical Analysis: Examples

- Example 2: `if (price>500) then tax=1xab`
  - Lexical analysis returns **ERROR** because 1xab is neither a number nor an identifier
  - Lexical analysis can help in reporting where an error occurs in the code
    - By recognizing '\n' as a token and incrementing the line number

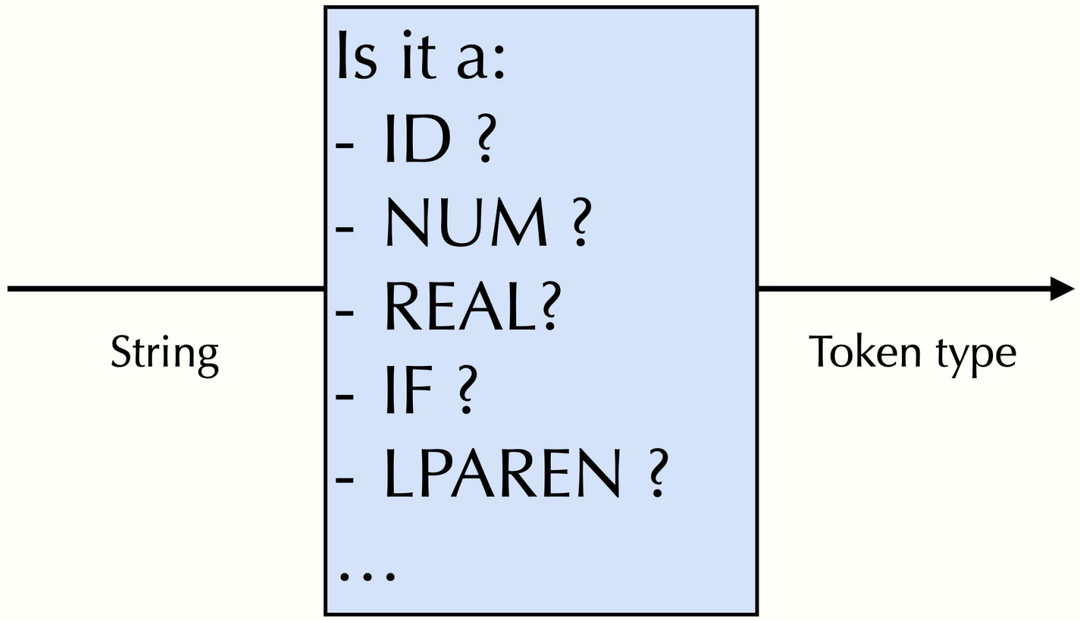
- Example 3: `if (price>500) thn tax=.08`
  - The lexical analysis returns:

```
IF LPAREN ID(price) GT NUM(500) RPAREN ID(thn) ID(tax) EQ REAL(0.08)
```

  - Is this an error at the level of lexical analysis?
    - No, it is an error at the level of syntax analysis!

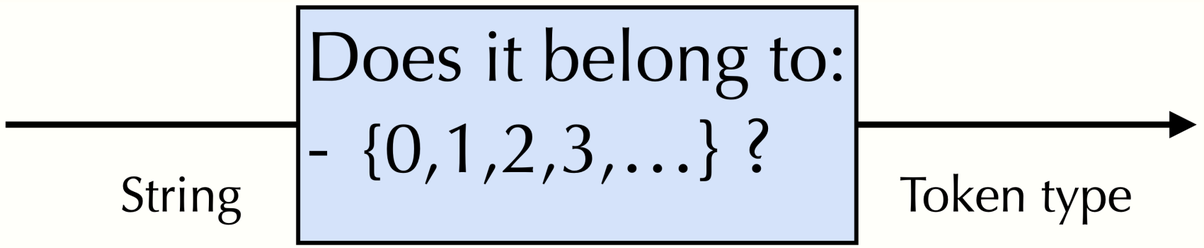
Towards Implementing a Lexical Analysis

- Recall: Lexical analysis breaks input into tokens.
- The lexical analysis needs to decide the token type for a given string (i.e., sequence of characters).



A Set Membership Question

- Recall: a token type specifies a set of acceptable tokens (i.e., strings).
  - The set of acceptable tokens for NUM is {0,1,2,3,…}.



  - But this set is **infinite**…

- How can we mechanically decide if a string belongs to a (possibly infinite) set S of strings?
  - Use a finite representation of S
    - **Regular expressions**
  - Check whether such a finite representation accepts the string
    - **Deterministic finite-state automata** (To be introduced in future lectures)
    - Use **OCamllex** to generate a lexer

Regular Expressions (RE)

Syntactic Sugar

- [0-9] shorthand for 0 | 1 | … | 9
- R? shorthand for (R | ε) (i.e., R is optional)
- R+ shorthand for (R R*) (i.e., at least one R)

❑RE Grammar

∅ (matches no string)	Concatenation $(R_1\ R_2)$
ε (epsilon – matches empty string)	Alternation $(R_1 \mid R_2)$
Literals (‘a’, ‘b’, ‘2’, ‘+’, etc.) drawn from alphabet	Kleene star $(R\ *)$

❑RE Semantics: each regular expression represents a set of strings

- $[\![\emptyset]\!] = \{\};\ [\![\varepsilon]\!] = \{\text{“”}\};\ [\![\text{’a’}]\!] = \{\text{“a”}\}$
- $[\![R_1\ R_2]\!] = \{s \mid s = \alpha\,\hat{}\,\beta \text{ and } \alpha \in [\![R_1]\!] \text{ and } \beta \in [\![R_2]\!]\}$
- $[\![R_1 \mid R_2]\!] = \{s \mid s \in [\![R_1]\!] \text{ or } s \in [\![R_2]\!]\} = [\![R_1]\!] \cup [\![R_2]\!]$
- $[\![\text{R}\ *]\!] = [\![\epsilon \mid \text{RR}\ *]\!] = \{s \mid s = \text{“”} \text{ or } s = \alpha\hat{}\beta \text{ and } \alpha \in [\![\text{R}]\!] \text{ and } \beta \in [\![\text{R}\ *]\!]\}$

❑Examples:

$(\ 0 \mid 1\ )*\ 0$	Binary numbers that are multiples of 2
$b*(abb*)*(a\|\varepsilon)$	Strings of a’s and b’s without consecutive a’s
$(a\|b)*aa(a\|b)*$	Strings of a’s and b’s with consecutive a’s

Regular Expressions to Specify Token Types

Reg Exp	Token Type
`if`	IF
`[a-z][a-z0-9]*`	ID
`[0-9]+`	NUM
`([0-9]+ "." [0-9]*) |` `([0-9]* "." [0-9]+)`	REAL

- Question: What is the token type of input "iffy"?
  - We want the token **ID(iffy)** rather than **IF**.
- In general, we want the longest match:
  - The longest initial substring of the input that can match a regular expression is taken as the next token

Generating a Lexer with OCamllex

- Ocamllex的词法分析器描述文件（see **Lexer.mll**）
  - `ocamllex Lexer.mll` 命令将产生词法分析器Lexer.ml
  - 使用OCaml 编译器转成可运行的词法分析器

```
{ 头部 OCaml 代码 }

let 名称 = 正则表达式

rule 函数名 = parse
   | 正则表达式 { OCaml 动作 }
   | 正则表达式 { OCaml 动作 }

{ 尾部 OCaml 代码 }
```

- **头部、尾部代码块**：原样复制到Lexer.ml的头部和尾部

- **解析规则**：规则将转化为一个函数，返回值为**{...}**的结果
  - 输入匹配哪个正则表达式，就返回对应{...}的结果
  - **{...}**的结果一般为匹配字符串的**Token类型**
  - `let **名称** = **正则表达式**`可以为正则表达式引入别名，后用于解析规则

- **lexbuf**: ocamllex自动提供的输入缓冲区参数
- **匹配原则**：优先最长匹配；长度相同时，选择位置靠前的规则

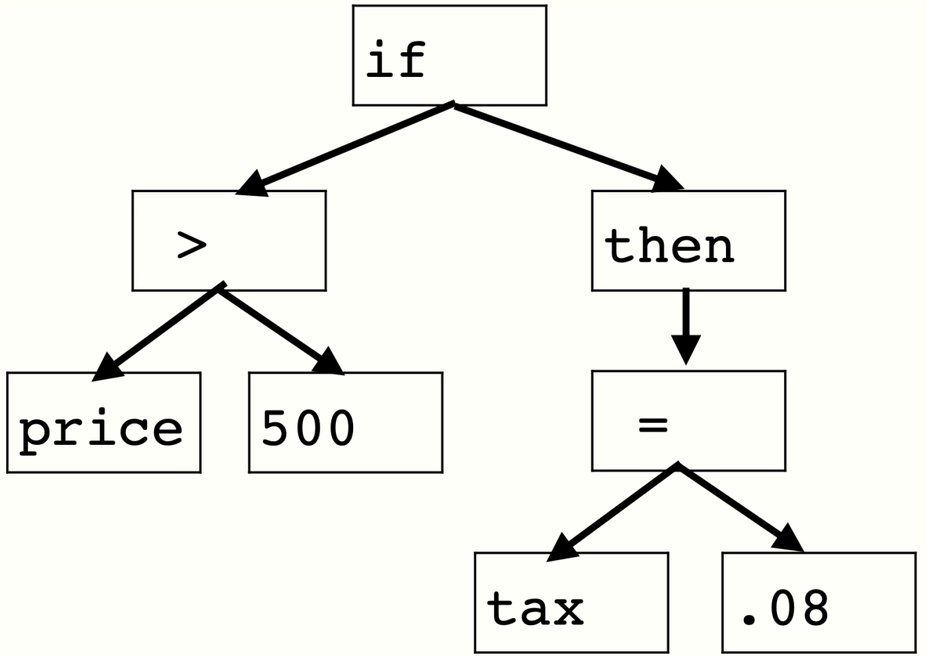
Generating a Lexer with OCamllex

- 我们支持的Token类型：
  - **关键字**：CLASS, EXTENDS, NEW, THIS, NULL, IF, ELSE, WHILE, FOR, RETURN, BREAK, CONTINUE, TRUE, FALSE, PRINT, PRINTLN, INT_TY, INT8_TY, BOOL_TY, VOID_TY
  - **运算符**：PLUS (+), MINUS (-), TIMES (*), DIV (/), REM (%), EQ (==), NE (!=), LT (<), LE (<=), GT (>), GE (>=), AND (&&), OR (||), NOT (!), BITAND (&), BITOR (|), BITXOR (^), BITNOT (~), SHL (<<), ASHR (>>)
  - **其他**：DOT (.), LPAREN ((), RPAREN ()), LBRACKET ([), RBRACKET (]), LBRACE ({), RBRACE (}), SEMI (;), COMMA (,), EOF (eof)
- 一些正则表达式： **digit** = [‘0’-‘9']; **integer_literal** = digit+; **letter** = ['a'-'z' 'A'-'Z’]; **identifier** = letter (letter | digit | '_’)*; **line_comment** = "//" [^ '\n’]*; whitespace = [' ' '\t' '\r’]+
- Demo: `cd lab4-AST`, `dune utop .`

```
-( 13:32:53 )-< command 0 >-------------------------------------------------------------------------
utop # open Lab4;;
-( 13:32:53 )-< command 1 >-------------------------------------------------------------------------
utop # Lexer.tokenize "while (i > 0) { i = i - 2; }";;
- : Tokens.token list =
[Tokens.WHILE; Tokens.LPAREN; Tokens.ID "i"; Tokens.GT; Tokens.INT 0L;
 Tokens.RPAREN; Tokens.LBRACE; Tokens.ID "i"; Tokens.ASSIGN; Tokens.ID "i";
 Tokens.MINUS; Tokens.INT 2L; Tokens.SEMI; Tokens.RBRACE; Tokens.EOF]
-( 13:33:00 )-< command 2 >-------------------------------------------------------------------------
utop #
```

Syntax Analysis

- Contains two pieces conceptually:
  - **Recognizing syntactically valid phrases**
    - Whether a token stream belongs to a language (e.g., C/C++/Java, etc)
    - Need to **specify the language**
  - **Extracting semantic content from the syntax**
    - E.g., What is the subject of the sentence?
    - E.g., What is the verb phrase?
    - E.g., Is the syntax ambiguous? If so, which meaning do we take?
      - "Time flies like an arrow" / "Fruit flies like a banana"
      - "2 * 3 + 4" / "x ^ f y"
    - Usually producing a parsing tree or an abstract syntax tree (AST)
  - In practice, solve both problems at the same time
- Example: `if price > 500 then tax = .08`
  - Syntax analysis produces a parsing tree (also called ST) or an AST



Specifying the Language

- A **language** is a set of strings. We need to specify what this set is.

- How about using regular expressions?
  - We can define sums of the form 4893 + 48 + 92 with
    - **sum = (digits "+")* digits**, where **digits = [0-9]+**
  - But what if we wanted to add parentheses to the language? 4893 + (48 + 92)
    - **sum = expr "+" expr**, **expr = digits | "(" sum ")”**, and **digits = [0-9]+**
  - A finite automaton cannot recognize the language of balanced parentheses!
    - try expanding the definition of **sum** in **expr**:
      - **expr = digits | "(" (digits | "(" expr "+" expr ")”) "+" ... ")”**
    - **Uh oh...**

- Use a more expressive grammar, i.e., **Context-Free Grammars** (CFGs)
  - Provide additional expressive power of **recursion**

Context-Free Grammars (CFGs)

- CFGs: regular expressions with **recursion**
  - CFGs provide a declarative specification of syntactic structure
- CFG has a set of **productions** of the form:
  - **`symbol → symbol symbol ... symbol`**
  - with zero or more symbols on the right
    - Each symbol is either **terminal** (i.e., a token from the alphabet)
    - or **non-terminal** (i.e., that appears on the LHS (Left-Hand Side) of some production)
  - The symbols that appear on the LHS of productions are Non-terminals

- CFG Example:
  - **Terminals**: **id print num , + ( ) := ;**
  - **Non-terminals**: S, E, L
  - **S** is the **start symbol**
  - The **language** specified by the grammar:
    - The set of **sentences** that can be derived from **S**
  - One sentence in the language:
    - **id := num; id := (id := num + num, id + id)**

$S \rightarrow S\texttt{;}\ S$	$E \rightarrow \texttt{id}$	$L \rightarrow E$
$S \rightarrow \texttt{id :=}\ E$	$E \rightarrow \texttt{num}$	$L \rightarrow L\texttt{,}\ E$
$S \rightarrow \texttt{print (}\ L\ \texttt{)}$	$E \rightarrow E + E$	
	$E \rightarrow \texttt{(}S\texttt{,}\ E\texttt{)}$	

Source text: “a := 7; b := (c := 30 + 5, a + c)”

Derivations and Parse Tree

- To show that a sentence is in the language of a grammar, we can perform a **derivation**
  - Start with the start symbol; repeatedly replace a non-terminal with its RHS (Right-Hand Side)

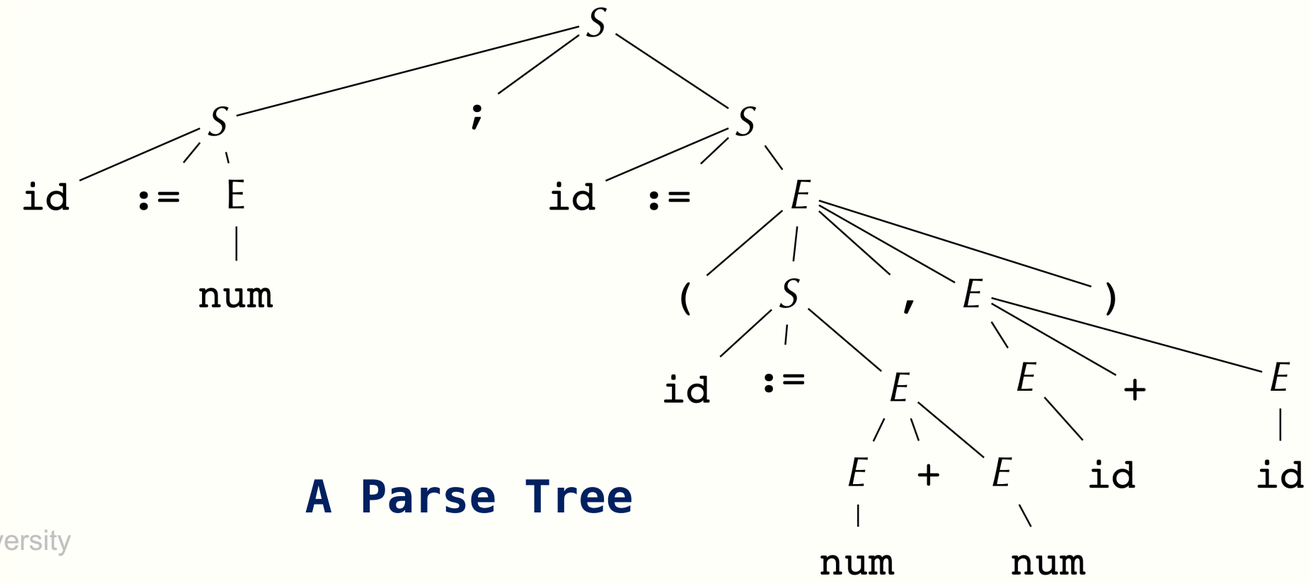
$S \rightarrow S$ `;` S	$E \rightarrow$ `id`	$L \rightarrow E$
$S \rightarrow$ `id :=` E	$E \rightarrow$ `num`	$L \rightarrow L$ `,` E
$S \rightarrow$ `print (` L `)`	$E \rightarrow E + E$	
	$E \rightarrow$ `(` S `,` E `)`	

Grammar

$S \rightarrow S; S$
$\rightarrow id := E; S$
$\rightarrow id := num;\ S$
$\rightarrow id := num; id := E$
$\rightarrow \ ...$
$\rightarrow \text{id} := \text{num};\ \text{id} := (\text{id} := \text{num} + \text{num}, \text{id} + \text{id})$

A Derivation

- A derivation constructs a **parse tree**
  - Also called **a syntax tree**
  - Different derivations may have the same tree

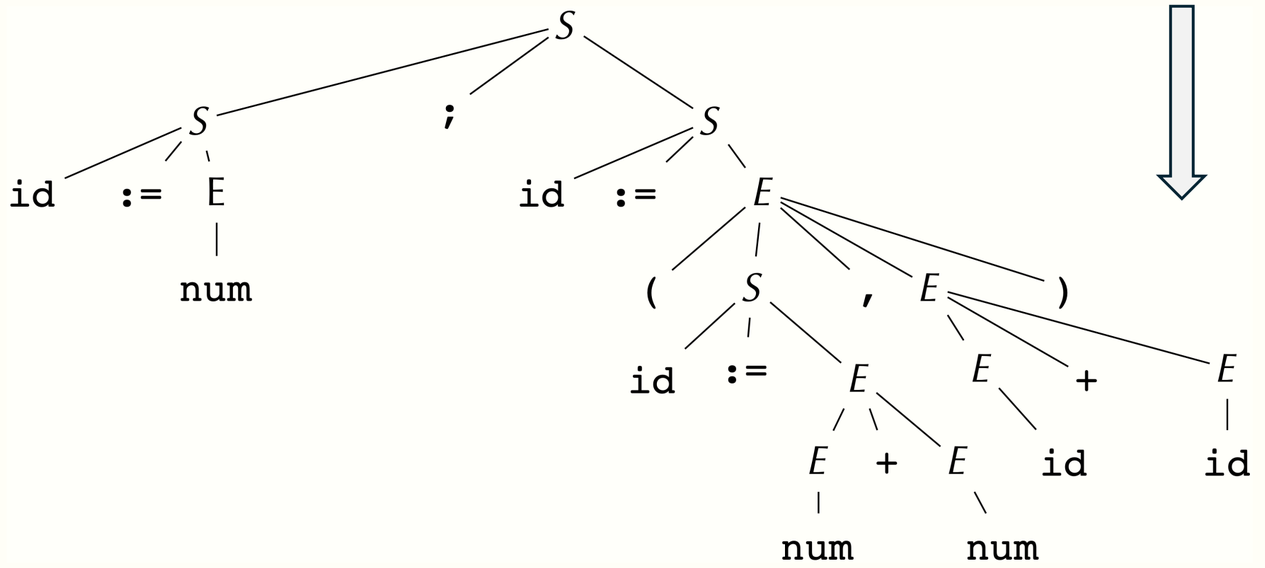


A Parse Tree

How to Build a Parse Tree/Find a Derivation

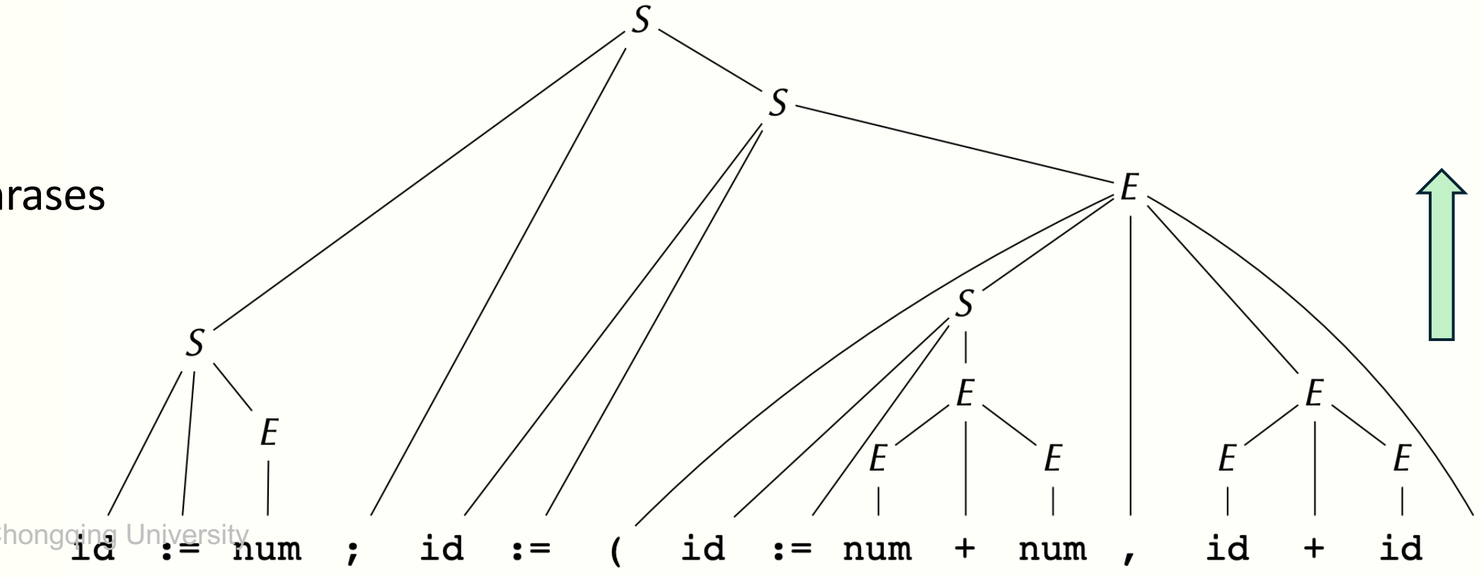
❑Top-down:

- Start from the start symbol, choose a non-terminal and expand until you reach the sentence



❑Bottom-up:

- Start from the terminals and replace phrases with non-terminals

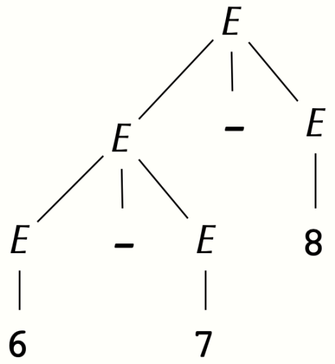


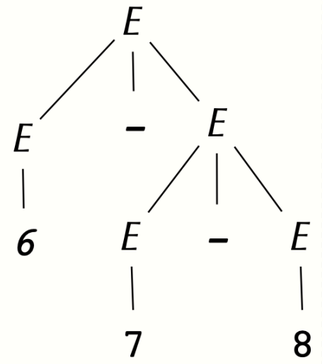
Dongjie He @ Chongqing University

Ambiguous Grammar

- An *ambiguous* grammar can derive different parse trees for a sentence
  - Ambiguity is usually bad: different parse trees often have different meanings!

$E \rightarrow \texttt{id}$
$E \rightarrow \texttt{num}$
$E \rightarrow E * E$
$E \rightarrow E \,/\, E$
$E \rightarrow E + E$
$E \rightarrow E - E$
$E \rightarrow (E)$



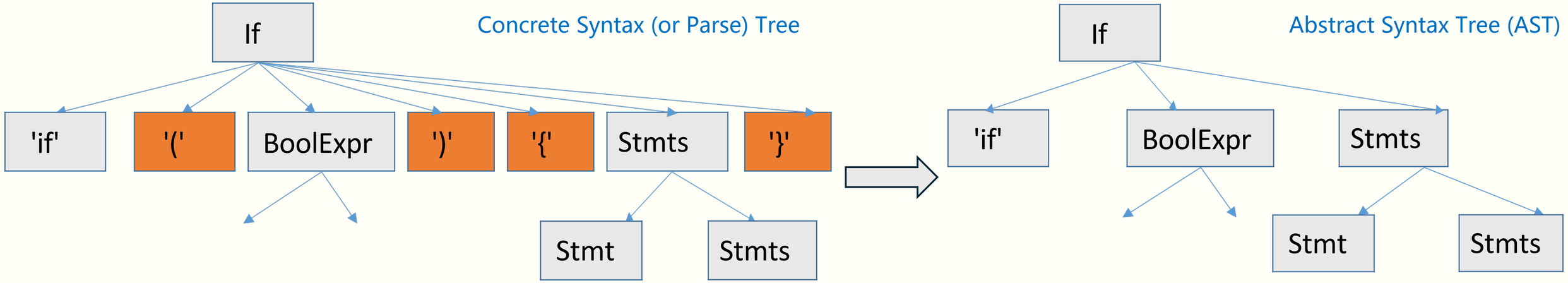


- Fixing Ambiguity:
  - We can usually **eliminate ambiguity by transforming the grammar**
  - * and / have higher **precedence** than + and –
    - 1 + 2 * 3 means 1+(2*3)
  - Operators have left **associativity**
    - 6-7-8 means (6-7)-8
  - E for expression, T for term, F for factor

$E \rightarrow E + T$	$T \rightarrow T * F$	$F \rightarrow \texttt{id}$
$E \rightarrow E - T$	$T \rightarrow T \,/\, F$	$F \rightarrow \texttt{num}$
$E \rightarrow T$	$T \rightarrow F$	$F \rightarrow (E)$

Concrete vs. Abstract Syntax Tree

- Syntax Tree:
  - 基于编程语言文法的源代码的树表示
- **Concrete Syntax**
  - The rules by which programs can be expressed as strings



- **Abstract Syntax**
  - Concerns only statements, expressions, and their operands
  - Don’t care about parentheses, semicolons, keywords, etc.

后续语义分析主要处理 AST，因此需要定义 AST，并在解析时构建它

Defining Our Toy Programming Language (LANG)

❑CFG Grammar for **Program Structure, Declarations, and Types**

```
program     ::= decl* EOF
decl        ::= class_decl | func_decl | global_decl

class_decl  ::= 'class' ID ('extends' ID)? '{' (field_decl | method_decl)* '}'
field_decl  ::= type ID ';'
method_decl ::= type ID '(' param_list? ')' '{' stmt* '}'
func_decl   ::= type ID '(' param_list? ')' '{' stmt* '}'    (* 顶层函数 *)
global_decl ::= type ID '=' exp ';'                  (* 全局变量 *)
param_list  ::= param (',' param)*
param       ::= type ID

base_type   ::= 'int' | 'int8' | 'boolean' | 'void' | ID
type        ::= base_type ('[' ']')*                 (* 任意层数的数组后缀 *)
cast_type   ::= ('int' | 'int8') ('[' ']')*      (* 语义检查仅允许 int / int8 *)
```

```
class Dog extends Animal {
  int8 breedId;
  void bark(int times) {
    /* statements */
  }
}
int globalCount = /* expression */;
void run(Animal a, int[] values) {
  /* statements */
}
```

An Example Code

Defining Our Toy Programming Language (LANG)

- CFG for **Expressions**

- Precedence and Associativity
  - 优先级由低到高

`||`	左结合
`&&`	左结合
`|`	按位或
`^`	按位异或
`&`	按位与
`== !=`	相等比较
`< <= > >=`	大小比较
`<< >>`	移位
`+ -`	左结合
`* / %`	左结合
`-!~`	右结合
`. () []`	字段访问、调用、下标）

```
exp ::= INT
      | 'true' | 'false' | 'null‘ | ID | 'this'
      | '-' exp | '!' exp | '~' exp            (* 一元 Neg / Not / BitNot *)
      | ‘(’ cast_type ‘)’ exp                   (* Cast; 仅整数目标类型通过语义检查 *)
      | exp '*' exp | exp '/' exp | exp '%' exp
      | exp '+' exp | exp '-' exp
      | exp '<<' exp | exp '>>' exp
      | exp '<' exp | exp '<=' exp | exp '>' exp | exp '>=' exp
      | exp '==' exp | exp '!=' exp
      | exp '&' exp | exp '^' exp | exp '|' exp
      | exp '&&' exp | exp '||' exp
      | 'new' ID '(' ')'                        (* New, 零参构造 *)
      | 'new' type '[' exp ']'                  (* NewArray, 类型为元素类型, 只分配这一层 *)
      | exp ‘[’ exp ‘]’                         (* ArrayGet *)
      | exp '.' ID                              (* FieldGet *)
      | exp '.' ID '(' arg_list? ')'            (* Call, 动态分派 *)
      | ID '(' arg_list? ')'                    (* CallFunc, 顶层函数调用 *)
      | '(' exp ')' '(' arg_list? ')'           (* 括号内须解析为 GlobalVar name *)
      | ('print' | 'println') '(' arg_list? ')' (* CallFunc, 内建函数调用 *)
      | '(' exp ')'

arg_list ::= exp (',' exp)*
```

Defining Our Toy Programming Language (LANG)

❑CFG Grammar for **Statements**

```
stmt ::= ';'                                    (* 空语句: Seq [] *)
       | '{' stmt* '}'                          (* 块语句: Seq of stmt list *)
       | local_decl                             (* 保存变量声明; 初始化转为 Assign *)
       | effect_stmt                            (* 赋值或表达式语句 *)
       | 'if' '(' exp ')' stmt ('else' stmt)?   (* 分支可为单语句或块 *)
       | 'while' '(' exp ')' stmt               (* While; 循环体可为单语句或块 *)
       | 'for' '(' for_init exp? ';' effect_node? ')' stmt
       | 'break' ';'                            (* Break; 跳出最近的循环 *)
       | 'continue' ';'                         (* Continue; 进入最近循环的下一轮 *)
       | 'return' exp? ';'                      (* Return*)

local_decl       ::= type ID '=' exp ';'
effect_stmt      ::= effect_node ';'
effect_node      ::= exp '=' exp | exp           (* 赋值左侧须为变量、字段或数组元素 *)
for_init         ::= ';' | local_decl | effect_statement
```

```
class Counter { int value; }

int doubleValue(int x) {
  return x + x;
}

void run() {
  Counter c = new Counter();
  int[] arr = new int[3];
  int i = 0;

  while (i < 3) {
    arr[i] = doubleValue(i);
    if (arr[i] > 2) {
      c.value = -arr[i];
    }
    i = i + 1;
  }
}
```

An Example Code

Defining the AST of Our Toy Programming Language

❑**AST.ml** 定义 AST 的结构，下一步让 Parser 按文法规则构建它

```
type program = {
  classes : class_decl list;
  funcs : func_decl list;
  globals : global_decl list;
}
```

```
type class_decl = {
  name : var;
  super : var option;
  fields : field list;
  methods : method_decl list;
}
type field = { fname : var; fty : ty }
type method_decl = func_decl
```

```
type func_decl = {
  name : var;
  params : var_decl list;
  locals : var_decl list;
  ret : ty;
  body : stmt;
}
type var_decl = { id : int;
name : var; ty : ty }
```

```
type global_decl = {
  gname : var;
  gty : ty;
  ginit : exp;
}
```

❑Expressions and Statements

```
type exp =
  | Int of int64 | Bool of bool | Null | Var of variable
  | This | Neg of exp | BitNot of exp | Cast of ty * exp
  | Binop of exp * binop * exp
  | Cmp of exp * cmpop * exp
  | And of exp * exp | Or of exp * exp | Not of exp
  | New of var | NewArray of ty * exp
  | ArrayGet of exp * exp | FieldGet of exp * var
  | Call of exp * var * exp list
  | CallFunc of var * exp list
type variable = LocalVar of int | GlobalVar of var
```

```
type stmt =
  | Seq of stmt list
  | Expr of exp
  | Assign of lvalue * exp
  | IfTE of exp * stmt * stmt
  | For of stmt * exp option * stmt * stmt
  | While of exp * stmt
  | Break | Continue
  | Return of exp option
type lvalue = LVar of variable
  | LField of exp * var | LIndex of exp * exp
```

Attribute Grammars

❑目标：用 Menhir 做语法制导翻译，在解析时构建 AST

- 已定义 AST 的数据结构，还需说明每条产生式如何组合子树
- 属性文法描述这种计算关系，Menhir 的语义动作执行这些计算

❑属性文法 = CFG + 属性 + 语义规则

- 属性携带 val、type 或 ast，语义规则定义它们的计算关系

属性	依赖来源	例：产生式与语义规则
综合属性 Synthesized	子结点的信息	$E \rightarrow E_1 + T$ $E.val = E_1.val + T.val$
继承属性 Inherited	父结点或兄弟结点的信息	$D \rightarrow T\ L$ $L.in = T.type$

❑本节聚焦 Synthesized-属性文法：只使用综合属性

S-Attributed Grammars

- **S = Synthesized：所有属性都是综合属性**
  - 对 A → X_1 … X_n，用右部符号的属性计算 A 的属性
- 例：加法与括号表达式，每个非终结符携带数值属性 val

产生式	语义规则
E → E_1 PLUS T	E.val = E_1.val + T.val
E → T	E.val = T.val
T → LPAREN E RPAREN	T.val = E.val
T → INT	T.val = INT.lexval

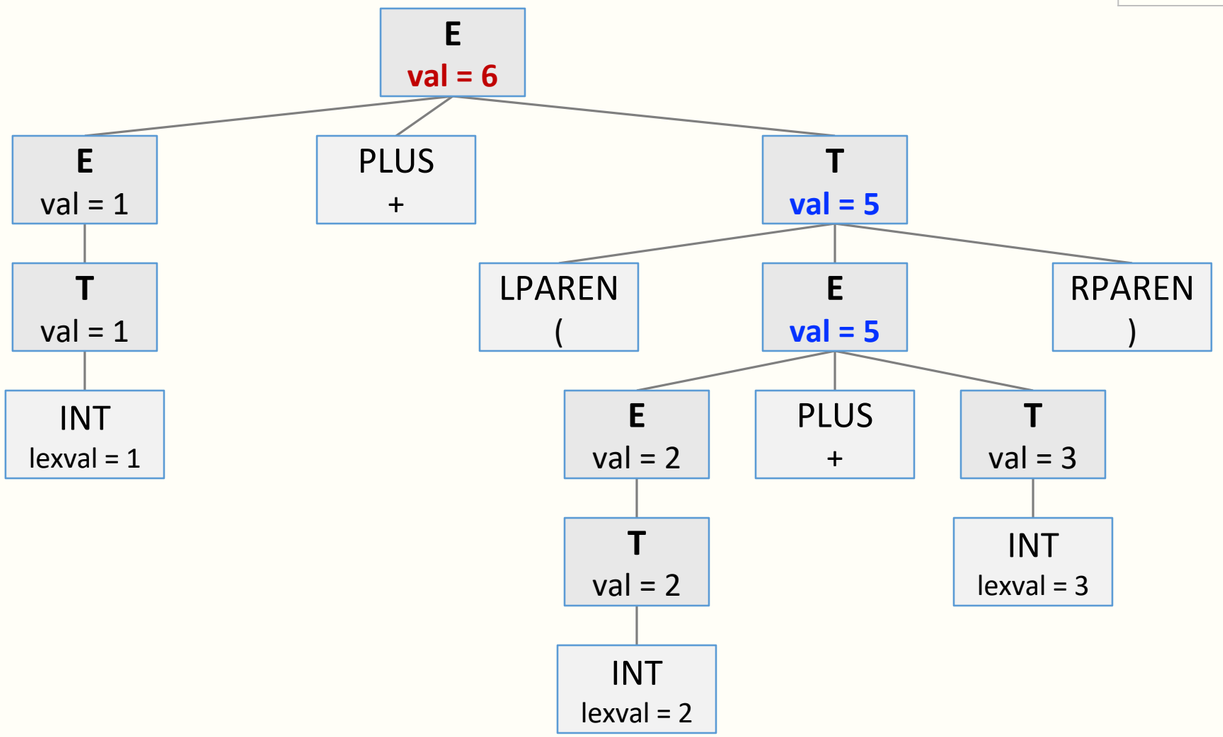
  - INT.lexval 由 lexer 提供。E_1 表示右部那一次出现的 E。

Evaluating S-Attributes

产生式	语义规则
E → E_1 PLUS T	E.val = E_1.val + T.val
E → T	E.val = T.val
T → LPAREN E RPAREN	T.val = E.val
T → INT	T.val = INT.lexval

❑先计算子结点，再计算父结点

- 输入：**1 + (2 + 3)**，计算根结点的 E.val



归约（reduce）：按产生式，
将匹配右部的符号串合成为左部非终结符。

子表达式	计算或传递属性
1	E.val = 1
2	E.val = 2
3	T.val = 3
2 + 3	**E.val = 2 + 3 = 5**
(2 + 3)	**T.val = E.val = 5**
1 + (2 + 3)	**E.val = 1 + 5 = 6**

❑归约时，右部符号的属性已经可用，语义动作据此计算左部属性。

Constructing an AST with S-Attributes

❑综合属性也可以是一棵树：把 val 换成 ast

产生式	语义规则
E → E_1 PLUS T	E.ast = Binop (E_1.ast, Add, T.ast)
E → T	E.ast = T.ast
T → LPAREN E RPAREN	T.ast = E.ast
T → INT	T.ast = Int (INT.lexval)

- 括号只决定组合结构，括号规则直接传递子树

1 + (2 + 3)

```
Binop (Int 1L, Add,
       Binop (Int 2L, Add, Int 3L))
```

Syntax-Directed Translation

❑语法制导翻译：按文法结构执行语义动作

- SDD（语法制导定义）说明属性怎样计算
- SDT（翻译方案）把动作嵌入产生式，明确何时执行

❑S-属性定义可实现为后缀 SDT：动作放在产生式末尾

$E \rightarrow E_1$ PLUS T
  { E.ast := Binop (E_1.ast, Add, T.ast) }
T → INT { T.ast := Int (INT.lexval) }

- 归约时执行末尾动作，将结果作为左部符号的语义值
- 可以边分析边构造 AST，无需先建立完整的具体语法树

S-Attributes in Menhir

❑Menhir 的 semantic value 对应这里的综合属性

```
%token <int64> INT
%token PLUS LPAREN RPAREN
%left PLUS
%type <AST.exp> expr
%%
expr:
  | n = INT { Int n }
  | a = expr; PLUS; b = expr { Binop (a, Add, b) }
  | LPAREN; e = expr; RPAREN { e }
```

属性文法中的概念	Parser.mly 中的实现
终结符的词法属性 INT.lexval	n = INT 取得 int64 值 n
右部非终结符的综合属性	a、b 是两棵已构造的 AST 子树
计算左部综合属性的规则	{ Binop (a, Add, b) } 返回左部 expr 的 AST

Generating a Parser with Menhir

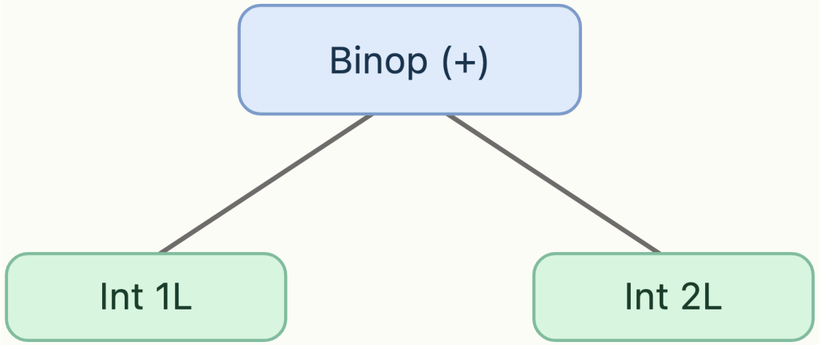
❑Menhir语法定义文件 (see Parser.mly)

- **Header** and **trailer** are arbitrary OCaml code, copied to the output file
  - trailer不需要在%{ %}之间，Menhir不推荐在Parser.mly中写trailer
- **Declarations** of
  - tokens: `%token <int64> INT`
  - start symbols: `%start <AST.program> parse_program`
  - OCaml types of symbols: ` %type <AST.exp> expr` (确保规则expr的语义动作返回类型为 AST.exp)
  - associativity and precedence of operators: ` %left AND `
- **Rules** are productions for nonterminals, with **semantic actions**
  - **Semantic actions** are OCaml expressions executed when grammar productions are reduced.
  - They compute **semantic values** for nonterminals.
  - They are commonly used to **construct AST nodes**.

```
%{
  header
%}
  declarations
%%
  rules
%%
  trailer
```

❑Grammer Rule Example：``1 + 2”

```
expr:
  | n = INT { Int n }
  | a = expr; PLUS; b = expr { Binop (a, Add, b) }
```



Generating a Parser with Menhir: Parser.mly

❑Whole structure of a program：

```
parse_program: declarations = list(declaration); EOF
  {
    let classes, funcs, globals = List.fold_right
        (fun decl (classes, funcs, globals) -> match decl with
          | `Class cls -> (cls :: classes, funcs, globals)
          | `Func func -> (classes, func :: funcs, globals)
          | `Global global -> (classes, funcs, global :: globals))
        declarations ([], [], []) in { AST.classes; funcs; globals }
  }
```

```
program      ::= decl* EOF
decl         ::= class_decl | func_decl | global_decl
```

```
declaration:
  | class_decl = class_declaration { `Class class_decl }
  | func = func_definition { `Func func }
  | gty = ty; gname = ID; ASSIGN; ginit = expression; SEMI
    { `Global { AST.gname; gty; ginit} }
```

❑Classes and function definitions

```
class_declaration: CLASS; name = ID; super = superclass; LBRACE; members = list(class_member); RBRACE
  {
    let fields, methods = List.fold_right
        (fun member (fields, methods) -> match member with
          | `Field field -> (field :: fields, methods)
          | `Method method_decl -> (fields, method_decl :: methods))
        members ([], []) in
    { AST.name; super; fields; methods}
  }
superclass: { None } | EXTENDS; name = ID { Some name }
class_member:
  | fty = ty; fname = ID; SEMI { `Field { AST.fname; fty } }
  | func = func_definition { `Method func }
```

```
(* Functions and methods share the same definition syntax. *)
func_definition:
  ret = ty; name = ID; LPAREN; params = parameters; RPAREN;
  LBRACE; stmts = list(statement); RBRACE
  {
    (* Parameters receive the first consecutive IDs, starting at zero. *)
    let count = List.length params in
    let locals = List.filter (fun (d : AST.var_decl) -> d.id >= count)
        (SymbolTable.symbols !(Context.symbols)) in
    Context.symbols := SymbolTable.create ();
    { AST.name; params; locals; ret; body = AST.Seq stmts }
  }
```

符号表确保函数中申明的每个变量具有唯一ID，我们稍后将介绍

Generating a Parser with Menhir: Parser.mly

- **Parser.mly：S-属性文法的一种实现形式**
  - 本课 AST 构造规则：读取右部语义值，返回左部的 AST 属性
- Generate a Parser:
  - menhir Parser.mly 生成 Parser.ml（实现）和 Parser.mli（接口）
  - menhir --explain ... 生成文法冲突的解释
- Demo: cd Lab4-AST, dune utop .
  - Use ASTPrinter.ml

```
int main() {
  int[] values = new int[2];
  values[0] = 42;
  return values[0];
}
```

Lexer & Parser

```
utop # open Lab4;;
─( 10:08:13 )─< command 1 >──────────────────────────────────{ counter: 0 }─
utop # Frontend.parse_program "int main() {  int[] values = new int[2];  values[
0] = 42;  return values[0];}" ;;
- : AST.program =
{AST.classes = [];
 funcs =
  [{AST.name = "main"; params = [];
    locals = [{AST.id = 0; name = "values"; ty = AST.TArray AST.TInt64}];
    ret = AST.TInt64;
    body =
     AST.Seq
      [AST.Assign (AST.LVar (AST.LocalVar 0),
        AST.NewArray (AST.TInt64, AST.Int 2L));
       AST.Assign (AST.LIndex (AST.Var (AST.LocalVar 0), AST.Int 0L),
        AST.Int 42L);
       AST.Return (Some (AST.ArrayGet (AST.Var (AST.LocalVar 0), AST.Int 0L)))]}
];
 globals = []}
─( 10:08:30 )─< command 2 >──────────────────────────────────{ counter: 0 }─
utop #
```

Name and Scope

- The same name in a program may refer to fundamentally different things:

```
public class A {
    char A;
    A A(A A) {
        A.A = 'A';
        return A((A) A);
    }
}
```

This is perfectly legal Java code.

```
int Awful() {
    int x = 137;
    {
        string x = "Scope!"
        if (float x = 0)
            double x = x;
    }
    if (x == 137) cout << "Y";
}
```

This is perfectly legal C++ code.

- The **scope** of an entity is the set of locations in a program where its name refers to it.
  - The introduction of new variables into scope may hide older variables.
- How do we keep track of what's visible?
  - **Symbol table**: A symbol table is a mapping from a name to the thing that name refers to

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x, y, z);
 4:   {
 5:     int x, z;
 6:     z = y;
 7:     x = z;
 8:     {
 9:       int y = x;
10:       {
11:         printf("%d,%d,%d\n", x, y, z);
12:       }
13:       printf("%d,%d,%d\n", x, y, z);
14:     }
15:     printf("%d,%d,%d\n", x, y, z);
16:   }
17: }
```

Symbol Table	
x	0
z	1

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x, y, z);
 4:   {
 5:     int x, z;
 6:     z = y;
 7:     x = z;
 8:     {
 9:       int y = x;
10:       {
11:         printf("%d,%d,%d\n", x, y, z);
12:       }
13:       printf("%d,%d,%d\n", x, y, z);
14:     }
15:     printf("%d,%d,%d\n", x, y, z);
16:   }
17: }
```

Symbol Table	
x	0
z	1
x	2
y	2

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x@2, y@2, z@1);
 4:   {
 5:     int x, z;
 6:     z = y;
 7:     x = z;
 8:     {
 9:       int y = x;
10:       {
11:         printf("%d,%d,%d\n", x, y, z);
12:       }
13:       printf("%d,%d,%d\n", x, y, z);
14:     }
15:     printf("%d,%d,%d\n", x, y, z);
16:   }
17: }
```

Symbol Table	
x	0
z	1
x	2
y	2

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x@2, y@2, z@1);
 4:   {
 5:     int x, z;
 6:     z@5 = y@2;
 7:     x@5 = z@5;
 8:     {
 9:       int y = x;
10:       {
11:         printf("%d,%d,%d\n", x, y, z);
12:       }
13:       printf("%d,%d,%d\n", x, y, z);
14:     }
15:     printf("%d,%d,%d\n", x, y, z);
16:   }
17: }
```

Symbol Table	
x	0
z	1
x	2
y	2
x	5
z	5

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x@2, y@2, z@1);
 4:   {
 5:     int x, z;
 6:     z@5 = y@2;
 7:     x@5 = z@5;
 8:     {
 9:       int y = x@5;
10:       {
11:         printf("%d,%d,%d\n", x@5, y@9, z@5);
12:       }
13:       printf("%d,%d,%d\n", x@5, y@9, z@5);
14:     }
15:     printf("%d,%d,%d\n", x, y, z);
16:   }
17: }
```

Symbol Table	
x	0
z	1
x	2
y	2
x	5
z	5
y	9

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x@2, y@2, z@1);
 4:   {
 5:     int x, z;
 6:     z@5 = y@2;
 7:     x@5 = z@5;
 8:     {
 9:       int y = x@5;
10:       {
11:         printf("%d,%d,%d\n", x@5, y@9, z@5);
12:       }
13:       printf("%d,%d,%d\n", x@5, y@9, z@5);
14:     }
15:     printf("%d,%d,%d\n", x@5, y@2, z@5);
16:   }
17: }
```

Symbol Table	
x	0
z	1
x	2
y	2
x	5
z	5

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x@2, y@2, z@1);
 4:   {
 5:     int x, z;
 6:     z@5 = y@2;
 7:     x@5 = z@5;
 8:     {
 9:       int y = x@5;
10:       {
11:         printf("%d,%d,%d\n", x@5, y@9, z@5);
12:       }
13:       printf("%d,%d,%d\n", x@5, y@9, z@5);
14:     }
15:     printf("%d,%d,%d\n", x@5, y@2, z@5);
16:   }
17: }
```

Symbol Table	
x	0
z	1
x	2
y	2

Symbol Tables: The Intuition

```
 0: int x = 137;
 1: int z = 42;
 2: int MyFunction(int x, int y) {
 3:   printf("%d,%d,%d\n", x@2, y@2, z@1);
 4:   {
 5:     int x, z;
 6:     z@5 = y@2;
 7:     x@5 = z@5;
 8:     {
 9:       int y = x@5;
10:       {
11:         printf("%d,%d,%d\n", x@5, y@9, z@5);
12:       }
13:       printf("%d,%d,%d\n", x@5, y@9, z@5);
14:     }
15:     printf("%d,%d,%d\n", x@5, y@2, z@5);
16:   }
17: }
```

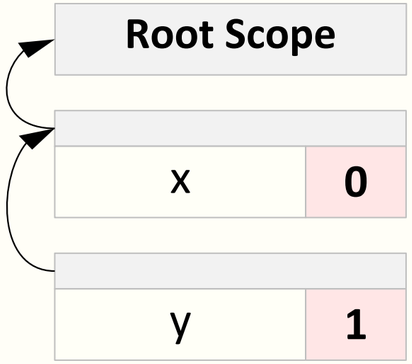
Symbol Table	
x	0
z	1

Symbol Table (SymbolTable.ml)

- Typically implemented as a **stack of maps**.
  - Each map corresponds to a particular scope.
  - Stack allows for easy “enter” and “exit” operations.
- Symbol Table Operations:
  - **Push scope**: Enter a new scope.
  - **Pop scope**: Leave a scope, discarding all declarations in it.
  - **Insert symbol**: Add a new entry to the current scope.
  - **Lookup symbol**: Find what a name corresponds to.
- Using a Symbol Table
  - To process a portion of the program that creates a scope (block statements, function calls, classes, etc.)
  - Enter a new scope.
  - Add all variable declarations to the symbol table.
  - Process the body of the block/function/class.
  - Exit the scope.

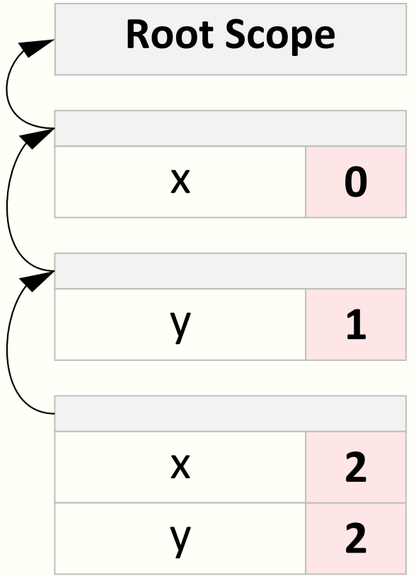
Another View of Symbol Tables

```
 0: int x;
 1: int y;
 2: int MyFunction(int x, int y)
 3: {
 4:   int w, z;
 5:   {
 6:     int y;
 7:   }
 8:   {
 9:     int w;
10:   }
11: }
```



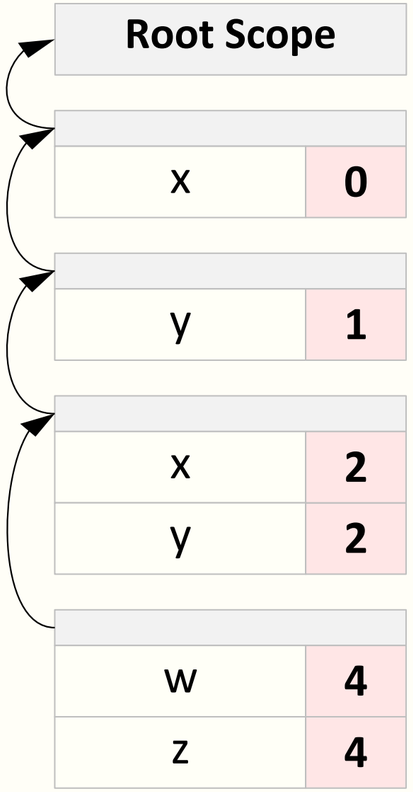
Another View of Symbol Tables

```
 0: int x;
 1: int y;
 2: int MyFunction(int x, int y)
 3: {
 4:   int w, z;
 5:   {
 6:     int y;
 7:   }
 8:   {
 9:     int w;
10:   }
11: }
```



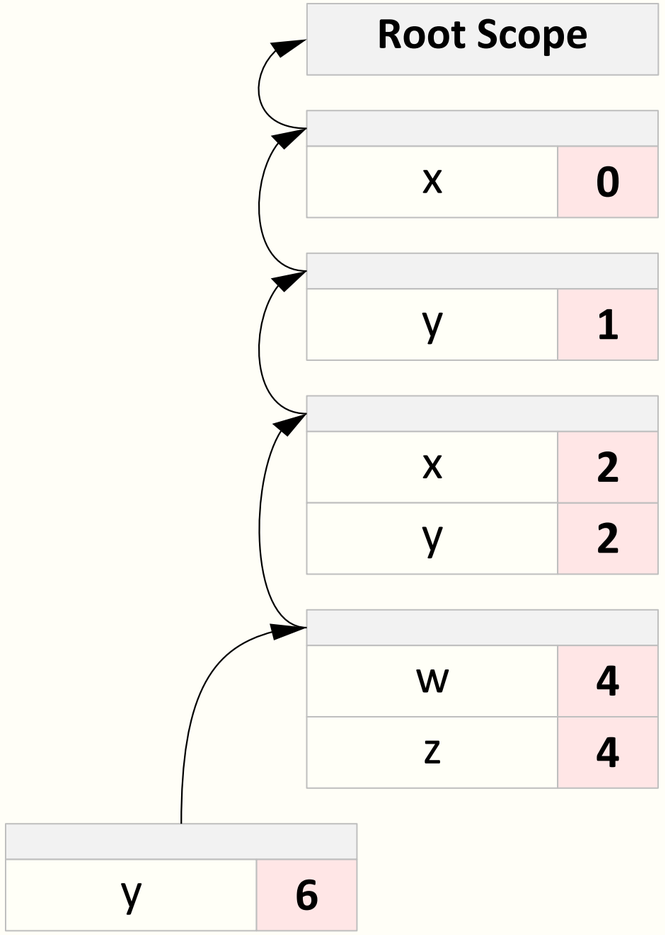
Another View of Symbol Tables

```
 0: int x;
 1: int y;
 2: int MyFunction(int x, int y)
 3: {
 4:   int w, z;
 5:   {
 6:     int y;
 7:   }
 8:   {
 9:     int w;
10:   }
11: }
```



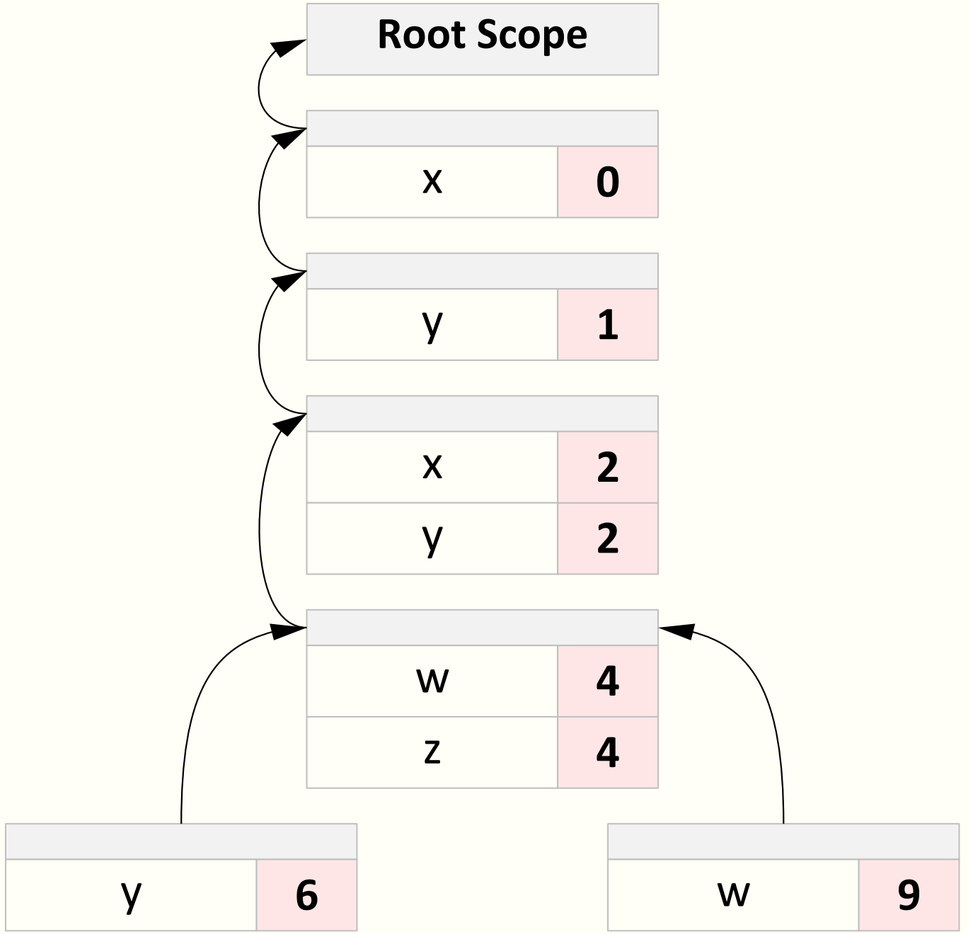
Another View of Symbol Tables

```
 0: int x;
 1: int y;
 2: int MyFunction(int x, int y)
 3: {
 4:   int w, z;
 5:   {
 6:     int y;
 7:   }
 8:   {
 9:     int w;
10:   }
11: }
```



Another View of Symbol Tables

```
 0: int x;
 1: int y;
 2: int MyFunction(int x, int y)
 3: {
 4:   int w, z;
 5:   {
 6:     int y;
 7:   }
 8:   {
 9:     int w;
10:   }
11: }
```



Spaghetti Stacks

- Treat the symbol table as a linked structure of scopes.
  - Each scope stores a pointer to its parents, but not vice-versa.
  - From any point in the program, symbol table appears to be a stack.

- Why two Interpretations?
  - Spaghetti stack more accurately captures the scoping structure.
  - Spaghetti stack is a *static* structure; explicit stack is a *dynamic* structure.
  - Explicit stack is an optimization of a spaghetti stack.

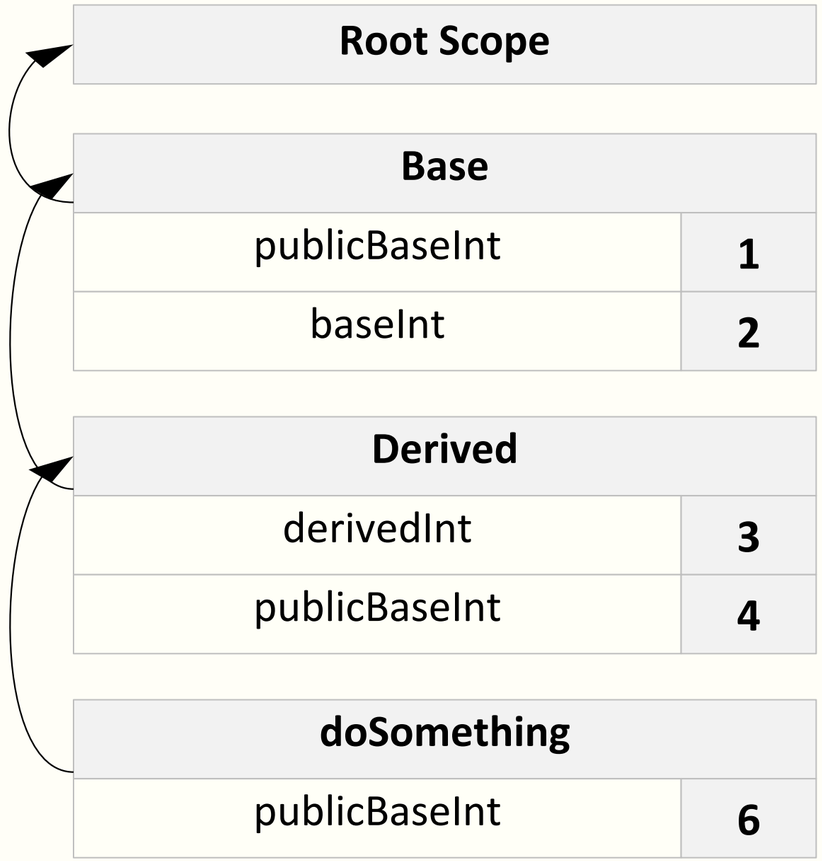
Scoping in Object-Oriented Languages

- Inheritance and Scoping
  - Typically, the scope for a derived class will store a link to the scope of its base class.
  - Looking up a field of a class traverses the scope chain until that field is found or a semantic error is found.

```
public class Base {
    public int publicBaseInt = 1;
    protected int baseInt = 2;
}
public class Derived extends Base {
    public int derivedInt = 3;
    public int publicBaseInt = 4;

    public void doSomething() {
        System.out.println(publicBaseInt);
        System.out.println(baseInt);
        System.out.println(derivedInt);
        int publicBaseInt = 6;
        System.out.println(publicBaseInt);
    }
}
```

```
> 4
  2
  3
  6
```



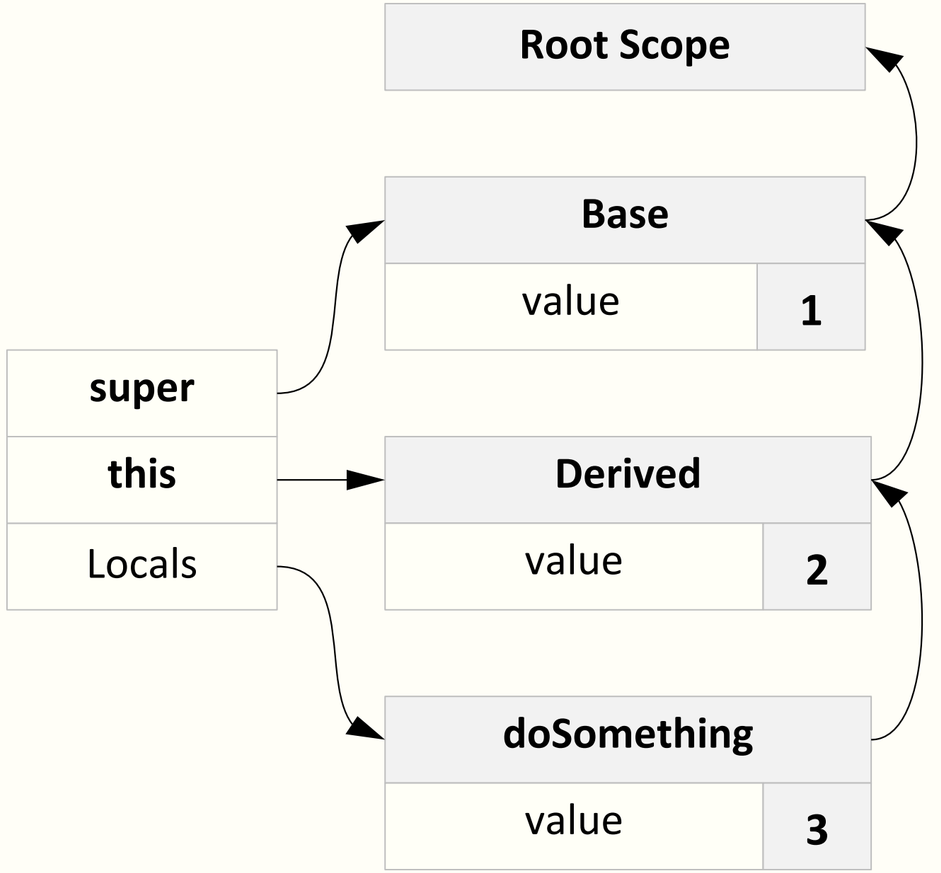
Explicit Disambiguation

```
public class Base {
    public int value = 1;
}

public class Derived extends Base {
    public int value = 2;

    public void doSomething() {
        int value = 3;
        System.out.println(value);
        System.out.println(this.value);
        System.out.println(super.value);
    }
}
```

```
> 3
  2
  1
```



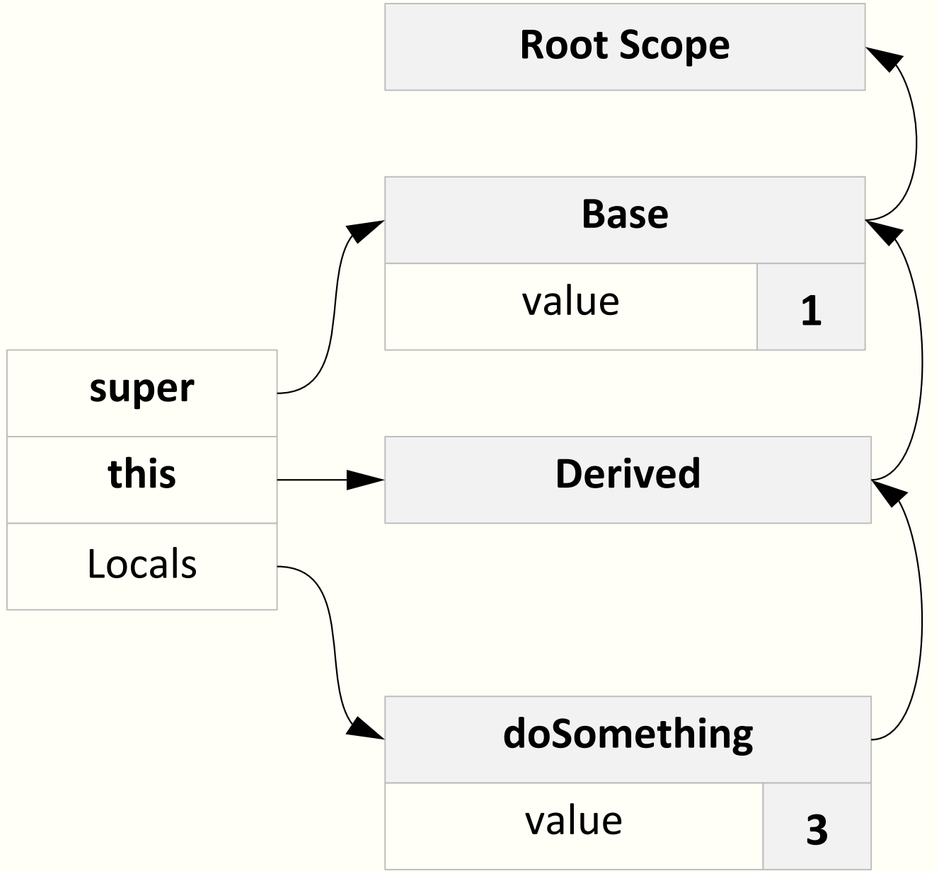
Explicit Disambiguation

```
public class Base {
    public int value = 1;
}

public class Derived extends Base {


    public void doSomething() {
        int value = 3;
        System.out.println(value);
        System.out.println(this.value);
        System.out.println(super.value);
    }
}
```

```
> 3
  1
  1
```



Disambiguating Scopes

- Maintain a second table of pointers into the scope stack.
- When looking up a value in a specific scope, begin the search from that scope.
- Some languages allow you to jump up to any arbitrary base class (for example, C++).

Scoping in Practice

```
class A {
public:
    /* … */

private:
    B* myB
};

class B {
public:
    /* … */

private:
    A* myA;
};
```

Error: B not declared

Scoping in C++

```
class A {
    private B myB;
};

class B {
    private A myA;
};
```

Perfectly fine!

Scoping in Java

Symbol tables and Our AST

- **Symbol tables构建成显式的Scope栈形式**
- **构建 AST 时，为每个参数和局部变量声明分配函数内唯一 ID**
  - 同一声明的使用共享 ID；不同声明即使同名，也使用不同 ID

```
type var_decl = { id : int; name : var; ty : ty }
type variable = LocalVar of int | GlobalVar of var
```

构建对象	AST 表示	作用
参数 / 局部声明	{ id; name; ty }	保存声明身份与类型
变量使用	LocalVar id / GlobalVar name	记录局部 ID 或待检查的全局名字

- **表达式用 Var (...) 包装；赋值左值用 LVar (...) 包装**

Declarations and Fresh IDs

- 处理声明时，只检查当前作用域是否已有同名声明
  - 检查通过后分配新 ID，登记 name 到 id 的映射，并保存 var_decl

声明处理（伪代码）

```
declare_local(name, ty):
  if lookup_current(name) != None:
    error("duplicate declaration")
  id = fresh_id()
  insert_current(name, id)
  return { id; name; ty }
```

声明所在位置	处理结果
当前层再次声明 x	立即报重复声明错误
内层首次声明 x	分配新 ID，允许遮蔽外层 x

- **查重只查当前层；内层与外层的同名声明使用不同 ID**
  - 退出作用域只移除活动映射，ID 计数器不回退

Resolving Variable Uses

- 使用变量时，从当前局部作用域向外逐层查找，采用第一个匹配
  - 查找范围包括函数参数和各层局部声明

变量使用的解析（伪代码）

```
resolve_variable(name):
  for scope in inner_to_outer(scopes):
    if scope.contains(name):
      return LocalVar(scope[name])
  return GlobalVar(name)
```

查找结果	返回的 variable
某一局部层命中 id	LocalVar id
所有局部层均未命中	GlobalVar name

- **局部栈未命中：保留名字，暂构造 GlobalVar name**
  - 后续语义检查验证它是否确实是已定义的全局变量

Example: Names to Variable IDs

- 参数 p、外层 x、内层 x、y 分别得到 id 0、1、2、3
  - 离开内层作用域后，新的 y 继续分配 id 3

```
int demo(int p) {  // p: id 0
  int x = p;       // id 1
  {
    int x = 2;     // id 2
    x = x + p;     // A
  }
  int y = x;       // id 3, B
  y = y + g;       // C
  return y;
}
```

位置	名字对应的 AST
A：x（左值）	LVar (LocalVar 2)
A：p	Var (LocalVar 0)
B：x	Var (LocalVar 1)
B：y（声明）	id = 3
C：y（左值）	LVar (LocalVar 3)
C：g	Var (GlobalVar "g")

- **A 使用内层 x；B 使用外层 x；g 暂记为全局变量**

Global References and Semantic Checking

- 构建 AST 先保留全局名字，语义检查阶段再验证其定义
- 语义检查先收集 program.globals，建立完整的全局声明表

```
Var (GlobalVar "g")
```

待检查引用	全局声明表	语义检查结果
GlobalVar "g"	已登记 g 的声明	绑定该声明，继续类型检查
GlobalVar "g"	找不到 g	报告未定义变量错误

- **GlobalVar "g" 是待验证的引用，不会自动创建全局声明**
- 局部重复声明在构建阶段报错；全局未定义在语义检查阶段报错

作业：正则表达式练习

- 写正则表达式：
  - 匹配一个6位数字的验证码，例如匹配038291，不匹配38291
  - 匹配仅由a、b构成，且至少含一个a的字符串，例如匹配abba，不匹配bbbb
  - 匹配简单邮箱地址：用户名由字母、数组、下划线组成，域名由字母和数组组成，后缀为.com或.cn, 例如匹配dongjiehe@cqu.edu.cn,不匹配abc@.com和abc@123.dom
  - 匹配IPV4地址，每段数字再0-255之间，不允许前导零，匹配192.168.1.1, 不匹配256.1.1.1
  - 匹配由 0 和 1 组成，且不能出现连续两个 1 的非空字符串，例如匹配10101，不匹配10110

- 下面这个问题可以用正则表达式表达吗？为什么？
  - 由{0,1}构成的回文串,如000, 1111, 1001,等.

作业：上下文无关文法练习

- 写上下文无关文法，识别如下语言：
  - $L = \{0^n 1^n | \ n \geq 0\}$
  - 所有正确匹配的括号串， 例如匹配(), (()), ()(), (()()), …， 不匹配)(, (()), ())(
  - $L = \{a^i b^j c^k | \ i, j, k \geq 0, i = j + k\}$
  - 二进制回文串，中间有分割符$L = \{\omega \ c \ \omega^R \mid \omega \in \{0, 1\}*\}$, ω^R表示ω的逆串
    - 匹配c, 0c0, 1c1, 01c10, 110c011, 不匹配0c1, 01c01, 10c10

- 下面语言可以用上下文无关文法识别吗？为什么？
  - $L = \{\omega \ \omega^R \mid \omega \in \{0, 1\}*\}$
  - $L = \{\omega \ \omega | \ \omega \in \{0, 1\}*\}$

作业：S-属性文法和AST遍历练习

1. S-属性文法：求值与 AST 构造

```
E → E '+' T | T
T → T '*' F | F
F → '(' E ')' | INT
```

- 写出各产生式的 val、ast 综合属性语义规则。

  INT.lexval 由词法分析器提供。

- 对 2 * (3 + 4) + 5：画出标注 val 的分析树，并给出最终 AST。

- 给出一次自底向上的属性计算顺序。

2. AST 遍历：补全 Printer

- 补全 ASTPrinter.ml 中下列构造子对应的递归打印分支：

  `Binop / Seq / Assign / While / LIndex`

- 沿用原有输出格式和缩进，按 AST 字段顺序遍历所有子结点。

测试片段（假设 a:int[]、i:int 已声明）：

```
while (i < 2) {
  a[i] = (i + 1) * 2;
  i = i + 1;
}
```

提交：第1题的语义规则、属性分析树和 AST；第2题的补全代码与测试输出。